# DySem: Uncovering Dynamic Semantic Components of Large Language Models for Calculating Semantic Textual Similarity

**Kaijie Zheng***, **Weiqin Wang***, **Yile Wang**✉, **Hui Huang**
Guangdong Provincial Key Laboratory of Visual Media and Multidimensional Intelligence
CSSE, Shenzhen University

## Abstract

Calculating semantic textual similarity is a foundational task in natural language processing. Current large language models (LLMs) based methods typically rely on extracting last-layer hidden states with fixed dimensions to compute similarity for every text pairs. We argue that this paradigm is suffer from two limitations: (i) The last hidden layer encodes more general knowledge rather than just semantic knowledge, making it suboptimal for semantic similarity computation; (ii) The hidden layer dimensions of LLMs are generally very large, which introduces some redundancy and noise for representing semantics. In this work, we propose DySem, a novel training-free framework that investigates more semantic-related internal components of LLMs via *multilingual consensus*, and shifts away from static representation spaces in favor of dynamic, sample-specific semantic dimensions by constructing text-dependent joint semantic set and computes similarity over this shared dimensional subset. Extensive experiments across various LLMs show that our method consistently outperforms recent baselines while maintaining lower dimensions for similarity calculation. The code is released at https://github.com/szu-tera/DySem.

## 1 Introduction

Evaluating the relationship between two texts and calculating their semantic textual similarity (STS; Agirre et al., 2012) plays an important role in natural language processing, serving as the backbones of applications such as semantic search, clustering, and information retrieval. Conventional pretrained models (PTMs; Devlin et al., 2019; Reimers and Gurevych, 2019; Gao et al., 2021) and large language models (LLMs; Jiang et al., 2024; Lei et al., 2024; Cheng et al., 2025) based methods typically map *all* text into a vector using the *same* last-layer hidden states with *fixed* dimensions for similarity calculation, as shown in Figure 1(a).

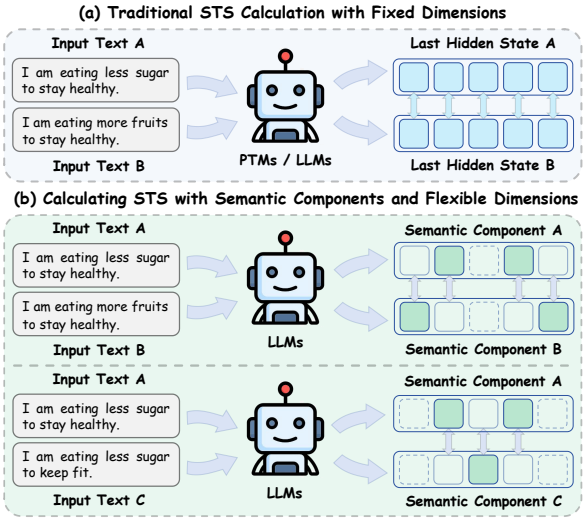



Figure 1: Illustration of different ways for calculating semantic similarity: (a) conventional methods use last hidden states and full dimensions; (b) our DySem investigates semantic components and flexible dimensions.

However, we argue that this default design has two limitations: First, the last-layer hidden states of LLMs typically encodes highly generalized information optimized for next-token prediction, which inevitably entangles core semantics with broader unrelated information. Studies also show that semantics are also captured by internal neurons of LLMs (Skean et al., 2025; Zhang et al., 2026). This motivates us to explore direct internal semantic components of LLMs for similarity computation.

Second, representing texts using the entire high-dimensional states can introduces information redundancy, and the semantic integrity is largely preserved after dimensionality reduction (Wang et al., 2023; Inkiriwang et al., 2025; Wang et al., 2025). Considering that the hidden state sizes of LLMs are often very large, this brings unnecessary overhead in textual similarity computation and downstream

*Equal contribution.
✉Corresponding to *wangyile@szu.edu.cn*.



*Proceedings of the 2026 Conference on Empirical Methods in Natural Language Processing*, pages 58948–58967
October 24-29, 2026 ©2026 Association for Computational Linguistics

applications. This motivates us to consider using a more flexible number of dimensions to compute the STS between two texts.

In this work, we propose DySem, a training-free method that aims to extract semantic components of LLMs beyond last-layer hidden states and shifts away from fixed representation dimensions in favor of dynamic, sample-specific semantic dimensions, as shown in Figure 1(b). In particular, DySem first extracts a sample-specific set of semantic components and dimensions via *multilingual consensus*, drawing from the straightforward intuition that a text's core semantic meaning remains invariant across its translated versions. Then, DySem dynamically construct a pair-dependent *joint semantic set* by taking the union of the semantic dimensions from both texts, and compute the STS over this joint semantic set. This dynamic alignment effectively filters out irrelevant background noise, ensuring that the metric evaluates the shared semantic concepts between texts.

Experiments on seven standard STS benchmarks across various LLMs show that, DySem can effectively uncovers attention-based semantic components of LLMs and demonstrates superior STS performance compared with recent baselines without model training. Moreover, DySem maintains flexible computational dimensions and can achieve competitive results using only around 1,000 dimensions or fewer on average across all samples.

## 2 Preliminaries and Notations

### 2.1 Internal Components of Transformer

Consider a decoder-only Transformer model $\mathcal{M}$ with layers $L$ and hidden size $d$. Let $N$ denotes the sequence length and $h_n^l \in \mathbb{R}^d$ denotes the hidden state of token $n$ at layer $l$, where $n \in \{0, 1, \ldots, N\}$, $l \in \{0, 1, \ldots, L\}$.

At layer $l$, the input state $h_n^{l-1}$ is first processed by the multi-head attention module and the attention output $a_n^l$ is calculated by:

$$a_n^l = \mathrm{MHA}^l(h_n^{l-1}), \tag{1}$$

where $\mathrm{MHA}^l$ denotes masked multi-head self-attention computed by the standard query-key-value mechanism. The resulting attention output $a_n^l$ is added by residual connection, and gives the output of feedforward network $f_n^l$:

$$f_n^l = \mathrm{FFN}^l(h_n^{l-1} + a_n^l). \tag{2}$$

The final hidden state $h_n^l$ at layer $l$ is calculated by the summation of the attention output, FFN output, and the hidden states from previous layer:

$$h_n^l = h_n^{l-1} + a_n^l + f_n^l, \tag{3}$$

where $a_n^l$ and $f_n^l$ represent the contributions of the MHA and FFN at layer $l$, respectively. Expanding the residual updates over all layers yields:

$$h_n^L = h_n^0 + \sum_{l=1}^{L} a_n^l + \sum_{l=1}^{L} f_n^l. \tag{4}$$

For convenience, define the cumulative attention and FFN contributions as:

$$A_n^L = \sum_{l=1}^{L} a_n^l, \qquad F_n^L = \sum_{l=1}^{L} f_n^l. \tag{5}$$

Thereby, the final-layer hidden state can be rewritten equivalently as:

$$h_n^L = h_n^0 + A_n^L + F_n^L. \tag{6}$$

For simplicity and clarity in our mechanistic analysis, we omit Layer Normalization (LayerNorm or RMSNorm) terms from these equations.

### 2.2 Semantic Textual Similarity Calculation

Given text pair $(x, y)$, conventional encoder-based methods such as SimCSE (Gao et al., 2021) perform contrastive learning with the hidden state $h_n^L$ of special token (e.g., [CLS]) as text representation and calculate their similarity using cosine distance:

$$s(x, y) = \cos(h_n^L(x), h_n^L(y)), \tag{7}$$

where $h_n^L$ can also be replaced by the pooling results of all hidden states $h_0^L, \ldots, h_n^L$.

To extract the text meaning from decoder-based LLMs without training, PromptEOL (Jiang et al., 2024) wraps each text $x$ with template $\mathcal{T}$ and extract the last hidden state $h_N^L$ as text representation:

$$\mathcal{M}(\mathcal{T}(x)) = [h_0^L, \ldots, h_N^L], \tag{8}$$

where $\mathcal{T}$ is used to extract text meanings as follows:

$$\mathcal{T} = \textit{This sentence: “x” means in one word:} \tag{9}$$

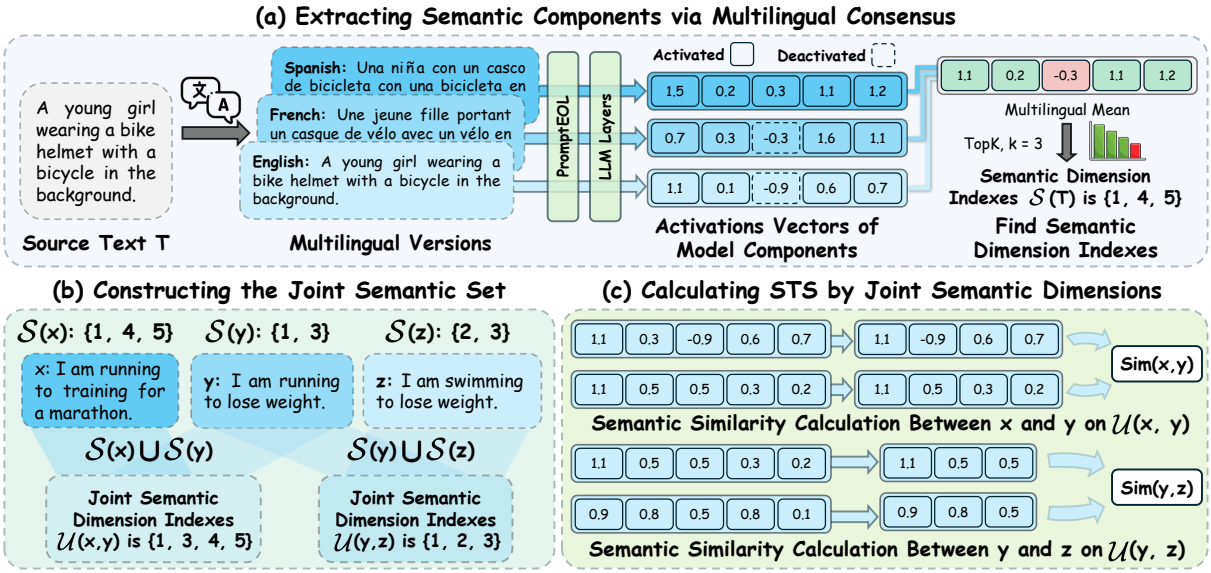



Figure 2: Overview of DySEM. The process involves (a) **extracting** semantic component dimension via multilingual consensus (§3.1), (b) **constructing** a pair-dependent joint semantic set $\mathcal{U}(x, y)$ by merging the individual semantic component dimension sets (§3.2), and (c) **calculating** the semantic similarity in the selected joint dimensions (§3.3).

**Our Motivations.** Whether using the hidden states from the special token or the last token, the above methods both use a fixed hidden state to represent the semantics of any given text $x$, and they utilize the full dimensionality of the states, i.e., the pre-defined hidden size $d$, when computing the semantic similarity. This motivates us to explore the following two questions. (1) **Representation Selection**: Can alternative internal components such as attention or FFN vectors in Eq. 4 and Eq. 5 be used for better representing semantics? (2) **Dimension Selection**: Do we really need two $d$-dimensional (typically 4096 or more for LLMs) vectors to compute textual similarity?

## 3 Method

Our method DySEM proceeds in three stages: (i) extracting *semantic components and dimensions* for each text via *multilingual consensus* (§3.1), (ii) constructing a pair-dependent *joint semantic set* by merging these individual dimensions (§3.2), and (iii) computing a *restricted cosine similarity* over the union of these dimension sets (§3.3). The overall framework is shown in Figure 2.

### 3.1 Extracting Semantic Components

Given a text $x$, we first translate it into a predefined set of $m$ target languages $\mathcal{L} = \{\ell_1, \ldots, \ell_m\}$, yielding a collection of language-specific renderings:

$$x^{(\ell)} = T_\ell(x), \quad \ell \in \mathcal{L}, \tag{10}$$

where $T_\ell$ is the translator with target language $\ell$.

To extract the semantic representation of $x$, we wrap each language version $x^{(\ell)}$ in the prompt template $\mathcal{T}$ to get their corresponding representations. We consider two prompt strategies: (1) **English Prompt**, which uses the same English template in Eq. 9 as the prompt for LLMs, and (2) **Language-Specific Prompt**, which uses translated version of the template $\mathcal{T}$ using the translator $T_\ell$ as the prompt. Then, we can get the internal state vector $\mathbf{s}^{(\ell)}(x) \in \mathbb{R}^d$ from the last-token, constructing a set of multilingual text vectors $\mathcal{E}(x) = \{\mathbf{s}^{(\ell)}(x) \mid \ell \in \mathcal{L}\}$. For the internal state vector $\mathbf{s}^{(\ell)}(x) \in \mathbb{R}^d$, we consider three components $h_n^L$, $A_n^L$, and $F_n^L$ as defined in §2.1. We analyze these components in §5.2.

We define *multilingual consensus* as the consistent activation patterns shared across these diverse linguistic renderings. By leveraging this consensus, we can effectively filter out language-specific surface artifacts and isolate the core text-relevant semantics. Specifically, this consensus is operationalized by identifying dimensions that are consistently activated across all language versions. Assuming a dimension is activated when its value is positive (Tang et al., 2024; Neo et al., 2024; Lee et al., 2025b), the consensus index set is defined as:

$$\mathcal{I}(x) = \left\{ j \;\middle|\; s_j^{(\ell)}(x) > 0,\ \forall\, \ell \in \mathcal{L} \right\}, \tag{11}$$

where $s_j^{(\ell)}(x)$ is the $j$-th component of $\mathbf{s}^{(\ell)}(x)$.

| | Methods | #Dim. | STS12 | STS13 | STS14 | STS15 | STS16 | STS-B | SICK-R | Avg. |
|---|---|---|---|---|---|---|---|---|---|---|
| | Sentence-BERT (110M)† (Reimers and Gurevych, 2019) | 768 | 70.97 | 76.53 | 73.19 | 79.09 | 74.30 | 77.03 | 72.91 | 74.89 |
| | SimCSE (110M)† (Gao et al., 2021) | 768 | 68.40 | 82.41 | 74.38 | 80.91 | 78.56 | 76.85 | 72.23 | 76.25 |
| | PromptBERT (110M)† (Jiang et al., 2022) | 768 | 71.56 | 84.58 | 76.98 | 84.47 | 80.60 | 81.60 | 69.87 | 78.54 |
| LLaMA2-7B | Mean Pooling | 4096 | 35.43 | 53.63 | 39.88 | 54.69 | 52.99 | 42.66 | 49.45 | 46.96 |
| LLaMA2-7B | PromptEOL† (Jiang et al., 2024) | 4096 | 58.80 | 77.00 | 66.34 | 73.22 | 73.53 | 71.72 | 70.19 | 70.11 |
| LLaMA2-7B | LLM2Vec (BehnamGhader et al., 2024) | 4096 | 65.39 | 79.26 | 72.98 | 82.72 | 81.02 | 78.32 | 71.77 | 75.92 |
| LLaMA2-7B | Token Prepending† (Fu et al., 2025) | 4096 | 66.90 | 83.12 | 74.31 | 79.87 | **80.03** | **80.67** | 75.40 | 77.19 |
| LLaMA2-7B | Contrastive Prompting†(Scaling; Cheng et al., 2025) | 4096 | 63.34 | 82.15 | 71.73 | 79.68 | 77.23 | 78.71 | 74.04 | 75.27 |
| LLaMA2-7B | Contrastive Prompting†(Recovering; Cheng et al., 2025) | 4096 | 63.37 | 81.95 | 71.90 | 79.54 | 77.29 | 78.36 | 74.00 | 75.20 |
| LLaMA2-7B | SemPA (Chen et al., 2026) | 4096 | 68.77 | 83.73 | 74.73 | 82.08 | 77.48 | 79.76 | **77.26** | 77.69 |
| LLaMA2-7B | AlignedWVA (Zhang et al., 2026) | 4096 | 67.94 | 84.97 | 75.93 | 81.68 | 79.98 | 80.39 | 74.45 | 77.91 |
| LLaMA2-7B | **DySEM (English Prompt, Ours)** | 1238* | 70.20 | **85.09** | **77.39** | 82.64 | 79.73 | **80.67** | 74.75 | 78.64 |
| LLaMA2-7B | **DySEM (Language-specific Prompt, Ours)** | 1240* | **72.68** | 84.75 | 77.28 | **83.47** | 79.19 | 80.06 | 75.39 | **78.98** |
| LLaMA3.1-8B | Mean Pooling | 4096 | 35.33 | 53.63 | 43.63 | 56.74 | 53.27 | 45.90 | 50.75 | 48.46 |
| LLaMA3.1-8B | PromptEOL (Jiang et al., 2024) | 4096 | 60.77 | 80.16 | 69.64 | 76.90 | 78.23 | 72.74 | 70.45 | 72.70 |
| LLaMA3.1-8B | AlignedWVA (Zhang et al., 2026) | 4096 | 64.13 | 83.98 | 72.66 | 80.41 | 80.64 | 78.91 | 72.99 | 76.25 |
| LLaMA3.1-8B | **DySEM (English Prompt, Ours)** | 1103* | 66.75 | 83.70 | 73.41 | 81.47 | 80.70 | 79.49 | 74.23 | 77.11 |
| LLaMA3.1-8B | **DySEM (Language-specific Prompt, Ours)** | 995* | **74.95** | **84.18** | **76.22** | **83.02** | **81.30** | **81.79** | **77.25** | **79.82** |
| Qwen2.5-7B | Mean Pooling | 3584 | 37.62 | 44.31 | 32.01 | 43.36 | 46.62 | 33.84 | 42.49 | 40.04 |
| Qwen2.5-7B | PromptEOL (Jiang et al., 2024) | 3584 | 65.50 | 79.70 | 70.62 | 79.05 | 78.73 | 76.44 | 70.46 | 74.36 |
| Qwen2.5-7B | AlignedWVA (Zhang et al., 2026) | 3584 | 71.56 | 80.19 | 69.61 | 75.63 | 78.69 | 74.45 | 73.06 | 74.74 |
| Qwen2.5-7B | **DySEM (English Prompt, Ours)** | 978* | **76.92** | **83.20** | **75.48** | **83.51** | 79.57 | **80.29** | **78.43** | **79.63** |
| Qwen2.5-7B | **DySEM (Language-Specific Prompt, Ours)** | 940* | 75.12 | 82.66 | 73.94 | 81.74 | **79.80** | 78.42 | 77.94 | 78.52 |
| Qwen3-8B | Mean Pooling | 4096 | 44.04 | 38.44 | 31.63 | 46.34 | 43.39 | 29.86 | 38.04 | 38.82 |
| Qwen3-8B | PromptEOL (Jiang et al., 2024) | 4096 | 59.30 | 74.19 | 61.56 | 72.36 | 74.64 | 73.41 | 66.90 | 68.91 |
| Qwen3-8B | AlignedWVA (Zhang et al., 2026) | 4096 | 64.91 | 80.90 | 70.56 | 77.38 | 78.90 | 76.15 | 74.75 | 74.79 |
| Qwen3-8B | **DySEM (English Prompt, Ours)** | 1027* | 75.22 | 83.22 | 76.82 | 83.49 | 79.54 | 80.09 | **79.16** | 79.65 |
| Qwen3-8B | **DySEM (Language-Specific Prompt, Ours)** | 675* | **77.54** | **84.33** | **76.95** | **83.91** | **81.19** | **81.26** | 78.56 | **80.54** |

Table 1: Comparison between baselines and our method on base models. In each column, the best results are in **bold**, and the second-best results are underlined. Our method's performance is highlighted in blue. † denotes results from the original paper. * indicates the average dimension of vectors across all samples for similarity computation.

Among the co-activated dimensions $\mathcal{I}(x)$, we rank each dimension by its mean activation across all languages:

$$\bar{s}_j(x) = \frac{1}{m} \sum_{\ell \in \mathcal{L}} s_j^{(\ell)}(x), \quad j \in \mathcal{I}(x), \tag{12}$$

and retain the top-$k$ dimensions with the highest mean activation:

$$\mathcal{S}(x) = \mathrm{TopKIdx}\Big(\{\bar{s}_j(x)\}_{j \in \mathcal{I}(x)}\Big), \tag{13}$$

where TopKIdx returns indices of the top $k$ values.

The index set $\mathcal{S}(x) \subset \{1, \ldots, d\}$ constitutes the *semantic component set* of text $x$: a compact summary of the dimensions most reliably encoding its meaning based on the multilingual consensus. An analysis of the choice of $k$ is provided in §5.6.

### 3.2 Constructing the Joint Semantic Set

The size of index set $\mathcal{S}(x)$ depends on the text $x$. For a given text pair $(x, y)$, to facilitate calculating their similarity practically, we construct *joint semantic set* as the union of their index set $\mathcal{S}(x)$ and $\mathcal{S}(y)$ from their semantic components:

$$\mathcal{U}(x, y) = \mathcal{S}(x) \cup \mathcal{S}(y). \tag{14}$$

This union preserves dimensions important to either text, allowing the comparison to capture both semantic overlap and difference. Since $\mathcal{U}(x, y)$ is derived directly from the text-specific sets $\mathcal{S}(x)$ and $\mathcal{S}(y)$, it is inherently determined dynamically for each text pair.

### 3.3 Calculating by Joint Semantic Dimensions

For any text $z$, let $\mathbf{v}(z)$ denote the internal state vector from the model. We further consider two variants of such vector: (i) using the source-language vector, and (ii) using the multilingual mean vector:

$$\mathbf{v}(z) = \begin{cases} \mathbf{s}^{(\ell_z)}(z), & \text{source}, \\ \frac{1}{m} \sum_{\ell \in \mathcal{L}} \mathbf{s}^{(\ell)}(z), & \text{mean}, \end{cases} \tag{15}$$

where $\ell_z$ is the source language of $z$.

We then construct text-specific joint semantic representation on $\mathcal{U}(x, y)$:

$$\mathbf{v}_{\mathcal{U}}(z) = \mathbf{v}(z)\big|_{\mathcal{U}(x,y)}, \quad z \in \{x, y\}, \tag{16}$$

where $\mathbf{v}(z)\big|_{\mathcal{U}(x,y)}$ selects the components indexed by $\mathcal{U}(x, y)$. The cosine similarity is computed as:

$$\mathrm{sim}(x, y) = \cos(\mathbf{v}_{\mathcal{U}}(x), \mathbf{v}_{\mathcal{U}}(y)). \tag{17}$$

Considering that the dimensions of vectors $\mathbf{v}_{\mathcal{U}}(x)$ and $\mathbf{v}_{\mathcal{U}}(y)$ are determined by the joint semantic set $\mathcal{U}(x, y)$ and depend on the input texts $x$ and $y$, thus the cosine similarity is computed over flexible dimensions.

| | Methods | #Dim. | STS12 | STS13 | STS14 | STS15 | STS16 | STS-B | SICK-R | Avg. |
|---|---|---|---|---|---|---|---|---|---|---|
| LLaMA2-7B-it | Mean Pooling | 4096 | 41.07 | 58.26 | 47.07 | 61.63 | 58.99 | 48.42 | 53.16 | 52.66 |
| | PromptEOL (Jiang et al., 2024) | 4096 | 59.19 | 78.18 | 67.77 | 75.57 | 71.35 | 72.69 | 70.45 | 70.74 |
| | AlignedWVA (Zhang et al., 2026) | 4096 | 60.95 | 79.83 | 69.90 | 80.56 | 75.57 | 76.97 | 75.58 | 74.20 |
| | **DySem (English Prompt, Ours)** | 968* | 64.07 | 79.62 | 71.91 | 82.04 | **76.01** | 77.99 | 75.99 | 75.38 |
| | **DySem (Language-Specific Prompt, Ours)** | 1193* | **70.84** | **80.52** | **74.85** | **82.69** | 74.61 | **79.23** | **77.39** | **77.16** |
| LLaMA3.1-8B-it | Mean Pooling | 4096 | 38.65 | 55.28 | 45.32 | 60.04 | 56.36 | 48.23 | 51.96 | 50.84 |
| | PromptEOL (Jiang et al., 2024) | 4096 | 62.27 | 78.30 | 69.53 | 76.32 | 78.80 | 74.99 | 71.97 | 73.17 |
| | AlignedWVA (Zhang et al., 2026) | 4096 | 64.89 | 80.89 | 71.97 | 81.04 | 82.35 | 80.28 | 75.67 | 76.73 |
| | **DySem (English Prompt, Ours)** | 1176* | 74.41 | 83.62 | 76.11 | 82.77 | 81.01 | 82.56 | 77.84 | 79.76 |
| | **DySem (Language-Specific Prompt, Ours)** | 772* | **76.38** | **84.87** | **77.23** | **83.92** | **82.50** | **83.77** | **79.74** | **81.20** |
| Qwen2.5-7B-it | Mean Pooling | 3584 | 41.56 | 45.39 | 33.67 | 44.40 | 50.63 | 35.35 | 43.60 | 42.09 |
| | PromptEOL (Jiang et al., 2024) | 3584 | 66.90 | 78.21 | 69.38 | 77.24 | 76.05 | 74.65 | 67.02 | 72.78 |
| | AlignedWVA (Zhang et al., 2026) | 3584 | 68.20 | 77.49 | 68.47 | 72.73 | 75.26 | 74.35 | 67.40 | 71.98 |
| | **DySem (English Prompt, Ours)** | 1001* | **77.54** | **82.28** | **75.35** | **83.49** | 78.89 | **80.49** | 78.08 | **79.44** |
| | **DySem (Language-Specific Prompt, Ours)** | 900* | 75.12 | 82.03 | 74.55 | 82.20 | **79.05** | 79.82 | **78.99** | 78.82 |
| Qwen3-8B-it | Mean Pooling | 4096 | 58.44 | 37.51 | 33.99 | 47.62 | 46.30 | 29.27 | 41.05 | 42.03 |
| | PromptEOL (Jiang et al., 2024) | 4096 | 50.62 | 66.34 | 55.32 | 68.43 | 75.46 | 72.63 | 64.44 | 64.75 |
| | AlignedWVA (Zhang et al., 2026) | 4096 | 63.73 | 79.34 | 68.70 | 76.55 | 79.05 | 77.25 | 73.29 | 73.99 |
| | **DySem (English Prompt, Ours)** | 1066* | 74.04 | 83.18 | 76.03 | 82.80 | 79.31 | 79.93 | 76.44 | 78.82 |
| | **DySem (Language-specific Prompt, Ours)** | 728* | **76.43** | **84.13** | **76.29** | **83.81** | **81.20** | **81.75** | **77.50** | **80.16** |
| Phi3-mini-it | Mean Pooling | 3072 | 58.57 | 52.75 | 45.73 | 46.02 | 50.30 | 39.33 | 41.99 | 47.81 |
| | PromptEOL (Jiang et al., 2024) | 3072 | 57.20 | 70.04 | 60.80 | 71.21 | 73.51 | 70.73 | 63.72 | 66.74 |
| | AlignedWVA (Zhang et al., 2026) | 3072 | 67.25 | 80.05 | 69.24 | 78.47 | **80.28** | 77.03 | 72.08 | 74.91 |
| | **DySem (English Prompt, Ours)** | 987* | 71.63 | 82.20 | 74.00 | 81.30 | 76.37 | 77.15 | 75.22 | 76.84 |
| | **DySem (Language-Specific Prompt, Ours)** | 869* | **72.39** | **82.41** | **74.02** | **82.81** | 79.98 | **81.20** | **76.77** | **78.51** |
| Phi3.5-mini-it | Mean Pooling | 3072 | 53.55 | 54.63 | 46.27 | 51.68 | 53.20 | 43.63 | 44.05 | 49.57 |
| | PromptEOL (Jiang et al., 2024) | 3072 | 63.30 | 75.42 | 65.57 | 73.81 | 76.04 | 72.85 | 68.33 | 70.76 |
| | AlignedWVA (Zhang et al., 2026) | 3072 | 66.35 | 78.00 | 68.61 | 78.05 | **79.47** | 76.82 | 74.26 | 74.51 |
| | **DySem (English Prompt, Ours)** | 997* | **71.87** | 81.40 | 73.49 | 80.48 | 77.27 | 78.10 | **77.36** | **77.14** |
| | **DySem (Language-Specific Prompt, Ours)** | 658* | 71.36 | **81.41** | **73.65** | **81.21** | 74.65 | **78.67** | 75.76 | 76.67 |

Table 2: Comparison between baselines and our method on instruct models.

# 4 Experiments

## 4.1 Experimental Settings

**Models.** We use ten LLMs covering both base and instruction-tuned variants from three families: (1) LLaMA (Touvron et al., 2023; Grattafiori et al., 2024): LLaMA2-7B, LLaMA2-7B-it, LLaMA3.1-8B, and LLaMA3.1-8B-it; (2) Qwen (Yang et al., 2024, 2025): Qwen2.5-7B, Qwen2.5-7B-it, Qwen3-8B, and Qwen3-8B-it; (3) Phi (Abdin et al., 2024): Phi3-mini-it, and Phi3.5-mini-it.

**Baselines.** We compare our method against pre-trained model based methods (Reimers and Gurevych, 2019; Gao et al., 2021; Jiang et al., 2022) and recent LLMs base methods including: **PromptEOL** (Jiang et al., 2024), **LLM2Vec** (BehnamGhader et al., 2024), **Token Prepending** (Fu et al., 2025), **Contrastive Prompting** (Cheng et al., 2025), **SemPA** (Chen et al., 2026), and **AlignedWVA** (Zhang et al., 2026). See Appendix A for introduction of these baselines.

**Datasets.** Seven STS datasets are evaluated, STS 2012∼2016 (Agirre et al., 2012, 2013, 2014, 2015, 2016), STS-B (Cer et al., 2017), and SICK-R (Marelli et al., 2014), see Appendix B for examples. We reported Spearman correlation (×100).

**Implementations.** For the main results, we use cumulative attention components $A_n^L$ in Eq. 5 and top-1024 semantic dimensions in Eq. 13 across all models. Both English prompt and language-specific prompt are used. See Appendix C for detailed settings of the vector selection in Eq. 15 and the specific number of languages used.

## 4.2 Main Results

The results are shown in Table 1 and 2, respectively. DySem achieves the best and second-best average results across all ten LLMs while requiring lower dimensions. Moreover, a paired two-tailed Student's $t$-test over all 70 model-dataset instances confirms that DySem significantly outperforms AlignedWVA ($t = 8.1948$, $p = 4.40 \times 10^{-12}$), demonstrating its effectiveness.

**Base Models.** For LLaMA2-7B base model, DySem achieves the highest average score of 78.98 among all compared methods. For Qwen3-8B, it further exhibits outstanding results of 80.54, surpassing AlignedWVA by 5.75 points and PromptEOL by 11.63 points. We also find that while achieving the best results, the dimensionality required by the model can be as low as 658, indicating that in the Qwen3-8B model, DySem

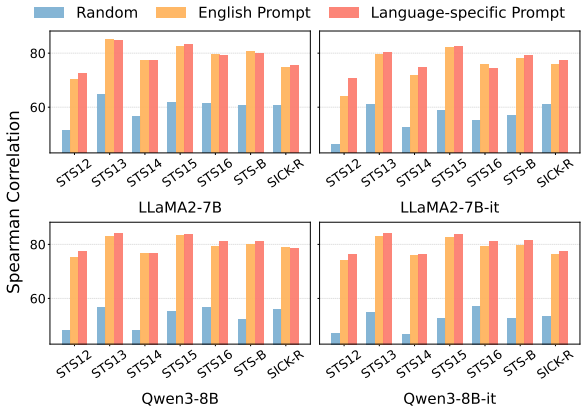



Figure 3: STS performance comparison between random components and semantic components.

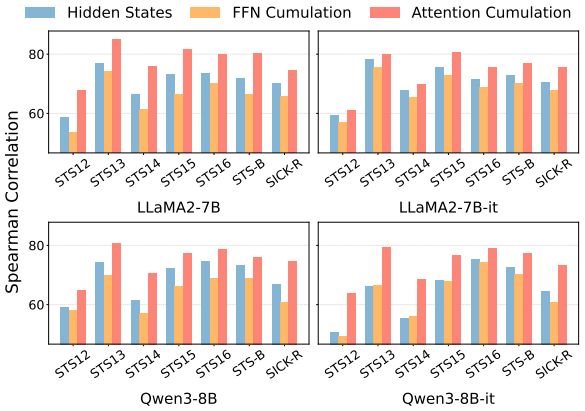



Figure 4: STS performance by selecting different internal components for semantic similarity calculation.

has indeed identified valuable internal components with significant semantic knowledge.

**Instruct Models.** A similar pattern holds on instruct models, where DySEM achieves the best average score in all settings and maintains a computational dimensionality of around 1,000 or fewer. Applying our method to LLaMA3.1-8B-it yields an average score of 81.20, with gains of 4.47 points over AlignedWVA and 8.03 points over PromptEOL. The improvement is also substantial on Qwen3-8B-it, where DySEM achieves an average score of 80.16 and outperforms AlignedWVA and PromptEOL by 6.17 and 15.41 points, respectively. Across instruction-tuned models, the average improvement over AlignedWVA is 4.55 points on instruct models, which is larger than the 3.82 point gain on base models. The reason can be that instruction tuning introduces more non-semantic variation inside models and DySEM is able to uncover more relevant semantic information to enable better semantic computation.

## 5 Analyses and Discussions

We further analyze the effectiveness of DySEM. Below we report the results using representative LLMs, and see Appendix D for additional ones.

### 5.1 Validity of Semantic Components

To verify the effectiveness of semantic components, we compare them with a random component baseline. The random baseline uniformly samples 1024 dimensions from the text representation. Results are shown in Figure 3.

The results show that semantic components consistently outperform random components across all 70 model-task comparisons, yielding a clear performance gap. On average, the random baseline scores only 52.70, whereas DySEM achieves 78.24 and 79.04 with English and language-specific prompts, respectively. This indicates that the gains of DySEM come from identifying meaningful semantic components rather than merely using a smaller subset of features.

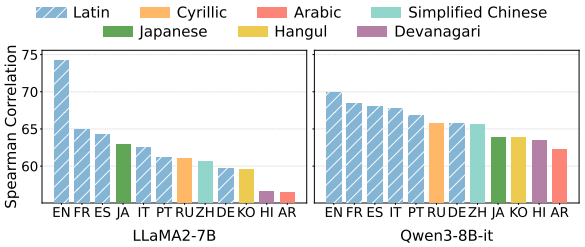



Figure 5: STS performance with texts in different candidate languages based on attention-based components.

### 5.2 Semantic Components Selection

To identify the best internal representation for STS calculation, we systematically evaluate three components across ten models: the final-layer hidden states ($h_n^L$), the cumulative FFN outputs ($F_n^L$), and the cumulative attention outputs ($A_n^L$).

Results are shown in Figure 4. The results demonstrate that $h_n^L$ is not universally optimal, while $A_n^L$ performs better in most cases. Across 70 comparisons spanning different models and tasks, $A_n^L$ achieves the best performance in 61 cases, while $h_n^L$ takes the lead in 9, and $F_n$ never ranks first. Overall, our analyses indicate that **attention-based representation $A_n^L$ is better suited to semantic similarity than the conventional default hidden states $h_n^L$**. The results reveal that $A_n^L$ could also capture the semantic-level features that can be

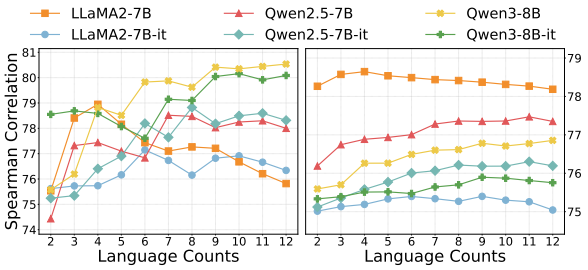



Figure 6: Comparison across different top-$m$ language selections on language-specific prompt with mean vector (left) and source vector (right).

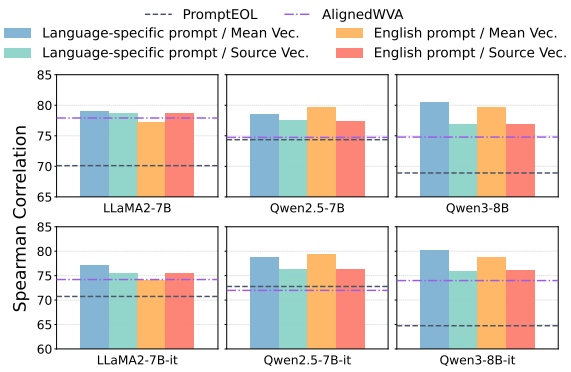



Figure 7: Comparison of English and language-specific prompts under source and mean vectors settings.

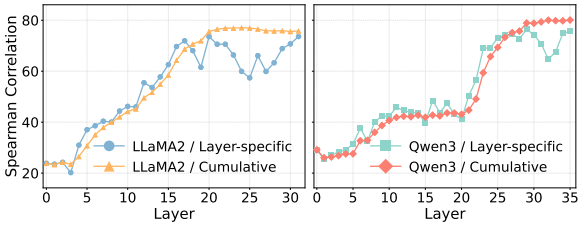



Figure 8: Comparison between layer-wise attention and cumulative attention aggregation strategies on LLaMA2-7B (left) and Qwen3-8B-it (right).

used for textual similarity computation.

### 5.3 Language Selection Strategies

To select the language set $\mathcal{L}$ in Eq. 10 for aggregation, we rank candidate languages based on their independent performance and retain the top-$m$ of them. Figure 5 shows that English consistently ranks first across models, with French and Spanish usually near the top. The reason can be that the model is more thoroughly trained on this type of Latin texts, resulting in a better understanding of them. We also find that the ordering of mid-tier languages varies substantially across models, so that language selection should be model-specific rather than relying on a fixed universal set.

Figure 6 illustrates how varying the pool size $m$ affects different configurations. Under the language-specific prompt and mean vector, performance is highly sensitive to $m$. Models like LLaMA2-7B peak early at $m = 4$, suggesting that incorporating lower-ranked languages introduces semantic noise. Conversely, models with stronger multilingual ability, such as Qwen3-8B, benefit from larger language pools. In contrast, the English prompt and source vector yields flatter performance curves. Together, these trends demonstrate that both careful language selection and the tuning of $m$ play a critical role in determining performance across various models and configurations.

### 5.4 Impacts of Prompts and Semantic Vector

We analyze the impact of English and language-specific prompts, alongside two semantic vector constructions, namely source and mean vectors. As illustrated in Figure 7, mean vector generally outperform source vector, especially under language-specific prompts, indicating that vector construction is the primary driver of performance.

The combination of a language-specific prompt and a mean vector achieves the highest average score on seven of the ten models. Notably, the choice of prompt language strongly interacts with vector construction. With the mean vector, language-specific prompts improve performance by 1.24 points on average, whereas with the source vector, prompt language differences are negligible, within 0.39 points. This suggests that language-specific prompts are most effective when translated-language representations are explicitly aggregated into the final semantic vector. Overall, our method significantly outperforms baselines across different settings, demonstrating that we indeed extracts more effective semantic features.

### 5.5 Comparing Two Attention Components

We further compare two ways to construct attention-based semantic components: (1) **layer-specific attention** $a_n^l$, which uses only the output of layer $l$, and (2) **cumulative attention** $A_n^l$, which accumulates outputs from the bottom layer up to layer $l$. The results with language-specific prompt and mean vector settings are shown in Figure 8.

Overall, the cumulative strategy outperforms the layer-specific strategy with more stable trends, especially in the middle-to-upper layers. For instance, peak performance over layer-specific at-

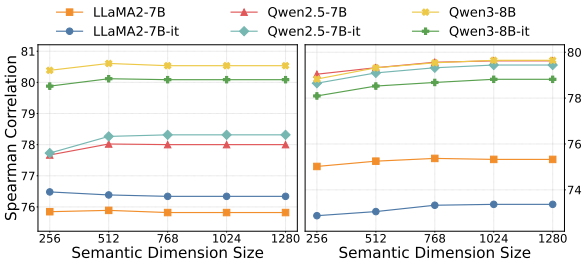



Figure 9: Analysis of top-$k$ semantic dimension selection on language-specific prompt (left) and English prompt (right) with mean vector .

tention rise from 76.43 to 80.09 on Qwen3-8B-it. These findings indicate that semantic information is distributed across layers rather than concentrated in one. We therefore use the final cumulative attention $A_n^L$ defined in §2.1 to better capture semantics.

### 5.6 Impacts of Semantic Dimension Sizes

We compare top-$k$ dimension selections for $k \in \{256, 512, 768, 1024, 1280\}$ in Eq. 13. As illustrated in Figure 9, performance patterns are primarily dictated by the prompt language. Under language-specific prompt settings, performance improves sharply up to $k = 512$ and then saturates.

By contrast, English prompt benefit from retaining significantly more dimensions, exhibiting an almost monotonic improvement up to $k = 1280$. These results indicate that semantic computation can be performed in lower dimensions, without necessarily using full dimensions of the hidden states (e.g., the typical size of 4096 for LLMs).

### 5.7 Analysis of Computational Overhead

We evaluate the computational overhead of DySEM using language-specific prompts, mean-vector aggregation, and four languages. As shown in Table 3, DySEM requires $4.21\times$ and $3.45\times$ the runtime of AlignedWVA on LLaMA2-7B and Qwen3-8B, respectively. This additional cost mainly arises from extracting and evaluating four language-specific representations. Nevertheless, the overhead enables DySEM to identify more compact and semantically informative dimensions, providing a practical trade-off between computational cost and representation quality.

### 5.8 Discussion on Further Applications

**Multi-modal Applications.** Although our dynamic semantic extraction framework is currently evaluated on text representation based on multilingual consensus, its core principle of identifying invariant dimensions across varied surface expressions can be naturally extended to broader vision domains. For example, treating images with operations like rotations or 3D point clouds with different camera angles as parallel "translations" can also used for extract shared semantic components.

Table 3: Runtime comparison between AlignedWVA and DySEM.

| Model | Method | Runtime (s) | Relative Runtime |
|---|---|---|---|
| LLaMA2-7B | AlignedWVA | 172.25 | $1.00\times$ |
| LLaMA2-7B | DySEM | 725.45 | $4.21\times$ |
| Qwen3-8B | AlignedWVA | 169.49 | $1.00\times$ |
| Qwen3-8B | DySEM | 585.33 | $3.45\times$ |

**Practical Deployments.** Our method can also be used for practical deployment such as text indexing and retrieval. By caching just the semantic dimension set $\mathcal{S}(y)$ alongside the representation vector for each database sample, the joint semantic set $\mathcal{U}(x, y)$ for a new query $x$ can be dynamically constructed on the fly. This allows for direct similarity computation without the computational overhead of re-encoding existing database samples. Ultimately, dynamic dimension selection serves as a lightweight and general processing step before similarity calculation and text retrieval.

## 6 Related Work

**Semantic Representation with LLMs.** Decoder-based LLMs originally target generation and recent works have leveraged them for extracting semantic representations. PromptEOL (Jiang et al., 2024) and MetaEOL (Lei et al., 2024) both extract text semantics solely through prompt templates, with the latter further introducing multi-perspective prompts to enrich representations. Fu et al. (2025) and Cheng et al. (2025) improve prompt-based LLM sentence embeddings through training-free inference-stage modifications, respectively by prepending decoded sentence embeddings to mitigate causal-attention limitations and by contrastively steering hidden states to emphasize core semantics over non-essential information. Recent embedding models, including NV-Embed (Lee et al., 2025a), Qwen3-Embedding (Zhang et al., 2025), and Gemini Embedding (Lee et al., 2025c), often rely on proprietary training data. These works rely on fixed, full-dimensional hidden states for text representation. Our work addresses these issues by directly mining internal LLM modules to extract

dynamic semantics in a training-free manner.

**Semantic Knowledge of Models.** Studies have shown that linguistic information is hierarchically organized across model layers: shallow layers predominantly capture surface-level and syntactic features, while deeper intermediate layers encode richer semantic and conceptual content (Jawahar et al., 2019; Ethayarajh, 2019; Liu et al., 2019). Recent evidence further suggests that intermediate representations can provide stronger feature quality than final-layer embeddings across a wide range of text-embedding tasks, indicating that semantic information is not confined to the output end of the model (Skean et al., 2025). Recent work by Zhang et al. (2026) shows that attention value vectors also encode some semantic knowledge. Inspired by them, we further investigate internal dynamic semantic component via multilingual consensus and novelly calculate semantic relationships with flexible dimensions, an aspect unexplored in prior work.

## 7 Conclusion

We propose DySem, a training-free framework for better calculating semantic textual similarity beyond the vanilla way of text representation which uses fixed final-layer hidden states and static full-dimensional vectors. DySem uncovers dynamic sample-specific semantic component via multilingual consensus, and uses flexible dimensions from semantic sets for similarity calculation. Extensive experiments demonstrate that DySem achieves superior performance across standard STS benchmarks, consistently outperforming a variety of base and instruction-tuned LLMs while maintaining lower dimensions. We hope that this work will provide new directions for better representing texts and evaluating their relationships with LLMs.

## Limitations

While our method demonstrates strong performance, extracting semantic dimensions introduces an $O(m)$ inference overhead due to the required $m$ translations and independent LLM forward passes. However, this cost is primarily concentrated in the offline database building phase; online queries remain highly efficient by dynamically constructing the joint semantic set $\mathcal{U}(x, y)$ from cached dimensions. Nonetheless, the initial encoding cost poses a challenge for massive-scale, cold-start applications compared to traditional single-pass encoders.

Additionally, our work thoroughly validates STS and conducts extensive experiments. Although STS directly measures text representation quality, future work can apply it on a broader suite of representation-heavy downstream tasks, such as large-scale information retrieval, text clustering, and zero-shot classification. Moreover, our framework primarily focuses on semantic computation between two texts, rather than high-quality embedding of a single text; therefore, we focus on extract semantics in a training-free way of LLMs and do not compare it with pure embedding models.

## Acknowledgments

This work was supported in part by ICFCRT (W2441020), Guangdong Basic and Applied Basic Research Foundation (2026A1515011358), Youth S&T Talent Support Programme of Guangdong Provincial Association for Science and Technology (SKXRC2026862), Shenzhen Natural Science Foundation (JCYJ20250604181610014), Intelligent Computing Center of Shenzhen University, and Scientific Development Fund from Guangdong Provincial Key Laboratory of Visual Media and Multidimensional Intelligence.

## References

Marah Abdin, Jyoti Aneja, Hany Awadalla, Ahmed Awadallah, Ammar Ahmad Awan, Nguyen Bach, Amit Bahree, Arash Bakhtiari, Jianmin Bao, Harkirat Behl, et al. 2024. Phi-3 technical report: A highly capable language model locally on your phone. *arXiv preprint arXiv:2404.14219*.

Eneko Agirre, Carmen Banea, Claire Cardie, Daniel Cer, Mona Diab, Aitor Gonzalez-Agirre, Weiwei Guo, Iñigo Lopez-Gazpio, Montse Maritxalar, Rada Mihalcea, German Rigau, Larraitz Uria, and Janyce Wiebe. 2015. SemEval-2015 task 2: Semantic textual similarity, English, Spanish and pilot on interpretability. In *Proceedings of the 9th International Workshop on Semantic Evaluation (SemEval 2015)*, pages 252–263, Denver, Colorado. Association for Computational Linguistics.

Eneko Agirre, Carmen Banea, Claire Cardie, Daniel Cer, Mona Diab, Aitor Gonzalez-Agirre, Weiwei Guo, Rada Mihalcea, German Rigau, and Janyce Wiebe. 2014. SemEval-2014 task 10: Multilingual semantic textual similarity. In *Proceedings of the 8th International Workshop on Semantic Evaluation (SemEval 2014)*, pages 81–91, Dublin, Ireland. Association for Computational Linguistics.

Eneko Agirre, Carmen Banea, Daniel Cer, Mona Diab, Aitor Gonzalez-Agirre, Rada Mihalcea, German

Rigau, and Janyce Wiebe. 2016. SemEval-2016 task 1: Semantic textual similarity, monolingual and cross-lingual evaluation. In *Proceedings of the 10th International Workshop on Semantic Evaluation (SemEval-2016)*, pages 497–511, San Diego, California. Association for Computational Linguistics.

Eneko Agirre, Daniel Cer, Mona Diab, and Aitor Gonzalez-Agirre. 2012. SemEval-2012 task 6: A pilot on semantic textual similarity. In **SEM 2012: The First Joint Conference on Lexical and Computational Semantics – Volume 1: Proceedings of the main conference and the shared task, and Volume 2: Proceedings of the Sixth International Workshop on Semantic Evaluation (SemEval 2012)*, pages 385–393, Montréal, Canada. Association for Computational Linguistics.

Eneko Agirre, Daniel Cer, Mona Diab, Aitor Gonzalez-Agirre, and Weiwei Guo. 2013. *SEM 2013 shared task: Semantic textual similarity. In *Second Joint Conference on Lexical and Computational Semantics (*SEM), Volume 1: Proceedings of the Main Conference and the Shared Task: Semantic Textual Similarity*, pages 32–43, Atlanta, Georgia, USA. Association for Computational Linguistics.

Parishad BehnamGhader, Vaibhav Adlakha, Marius Mosbach, Dzmitry Bahdanau, Nicolas Chapados, and Siva Reddy. 2024. LLM2vec: Large language models are secretly powerful text encoders. In *First Conference on Language Modeling*.

Daniel Cer, Mona Diab, Eneko Agirre, Iñigo Lopez-Gazpio, and Lucia Specia. 2017. SemEval-2017 task 1: Semantic textual similarity multilingual and crosslingual focused evaluation. In *Proceedings of the 11th International Workshop on Semantic Evaluation (SemEval-2017)*, pages 1–14, Vancouver, Canada. Association for Computational Linguistics.

Ziyang Chen, Zhenxuan Huang, Yile Wang, Weiqin Wang, Lu Yin, and Hui Huang. 2026. SemPA: Improving sentence embeddings of large language models through semantic preference alignment. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 37299–37312, San Diego, California, United States. Association for Computational Linguistics.

Zifeng Cheng, Zhonghui Wang, Yuchen Fu, Zhiwei Jiang, Yafeng Yin, Cong Wang, and Qing Gu. 2025. Contrastive prompting enhances sentence embeddings in LLMs through inference-time steering. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3475–3487, Vienna, Austria. Association for Computational Linguistics.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4171–4186, Minneapolis, Minnesota. Association for Computational Linguistics.

Kawin Ethayarajh. 2019. How contextual are contextualized word representations? Comparing the geometry of BERT, ELMo, and GPT-2 embeddings. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 55–65, Hong Kong, China. Association for Computational Linguistics.

Yuchen Fu, Zifeng Cheng, Zhiwei Jiang, Zhonghui Wang, Yafeng Yin, Zhengliang Li, and Qing Gu. 2025. Token prepending: A training-free approach for eliciting better sentence embeddings from LLMs. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3168–3181, Vienna, Austria. Association for Computational Linguistics.

Tianyu Gao, Xingcheng Yao, and Danqi Chen. 2021. SimCSE: Simple contrastive learning of sentence embeddings. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 6894–6910, Online and Punta Cana, Dominican Republic. Association for Computational Linguistics.

Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*.

Nathan Inkiriwang, Necva Bölücü, Garth Tarr, and Maciej Rybinski. 2025. Do we really need all those dimensions? an intrinsic evaluation framework for compressed embeddings. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 13305–13323, Suzhou, China. Association for Computational Linguistics.

Ganesh Jawahar, Benoît Sagot, and Djamé Seddah. 2019. What does BERT learn about the structure of language? In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 3651–3657, Florence, Italy. Association for Computational Linguistics.

Ting Jiang, Shaohan Huang, Zhongzhi Luan, Deqing Wang, and Fuzhen Zhuang. 2024. Scaling sentence embeddings with large language models. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 3182–3196.

Ting Jiang, Jian Jiao, Shaohan Huang, Zihan Zhang, Deqing Wang, Fuzhen Zhuang, Furu Wei, Haizhen Huang, Denvy Deng, and Qi Zhang. 2022. PromptBERT: Improving BERT sentence embeddings with prompts. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 8826–8837, Abu Dhabi, United Arab Emirates. Association for Computational Linguistics.

Chankyu Lee, Rajarshi Roy, Mengyao Xu, Jonathan Raiman, Mohammad Shoeybi, Bryan Catanzaro, and Wei Ping. 2025a. NV-embed: Improved techniques for training LLMs as generalist embedding models. In *The Thirteenth International Conference on Learning Representations*.

Jaewook Lee, Junseo Jang, Oh-Woog Kwon, and Harksoo Kim. 2025b. Small changes, big impact: How manipulating a few neurons can drastically alter LLM aggression. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 23478–23505, Vienna, Austria. Association for Computational Linguistics.

Jinhyuk Lee, Feiyang Chen, Sahil Dua, Daniel Cer, Madhuri Shanbhogue, Iftekhar Naim, Gustavo Hernández Ábrego, Zhe Li, Kaifeng Chen, Henrique Schechter Vera, et al. 2025c. Gemini embedding: Generalizable embeddings from gemini. *arXiv preprint arXiv:2503.07891*.

Yibin Lei, Di Wu, Tianyi Zhou, Tao Shen, Yu Cao, Chongyang Tao, and Andrew Yates. 2024. Meta-task prompting elicits embeddings from large language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 10141–10157.

Nelson F. Liu, Matt Gardner, Yonatan Belinkov, Matthew E. Peters, and Noah A. Smith. 2019. Linguistic knowledge and transferability of contextual representations. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 1073–1094, Minneapolis, Minnesota. Association for Computational Linguistics.

Marco Marelli, Stefano Menini, Marco Baroni, Luisa Bentivogli, Raffaella Bernardi, and Roberto Zamparelli. 2014. A SICK cure for the evaluation of compositional distributional semantic models. In *Proceedings of the Ninth International Conference on Language Resources and Evaluation (LREC'14)*, pages 216–223, Reykjavik, Iceland. European Language Resources Association (ELRA).

Clement Neo, Shay B Cohen, and Fazl Barez. 2024. Interpreting context look-ups in transformers: Investigating attention-MLP interactions. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 16681–16697, Miami, Florida, USA. Association for Computational Linguistics.

Nils Reimers and Iryna Gurevych. 2019. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 3982–3992, Hong Kong, China. Association for Computational Linguistics.

Oscar Skean, Md Rifat Arefin, Dan Zhao, Niket Nikul Patel, Jalal Naghiyev, Yann LeCun, and Ravid Shwartz-Ziv. 2025. Layer by layer: Uncovering hidden representations in language models. In *Forty-second International Conference on Machine Learning*.

Tianyi Tang, Wenyang Luo, Haoyang Huang, Dongdong Zhang, Xiaolei Wang, Xin Zhao, Furu Wei, and Ji-Rong Wen. 2024. Language-specific neurons: The key to multilingual capabilities in large language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 5701–5715, Bangkok, Thailand. Association for Computational Linguistics.

Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*.

Hongwei Wang, Hongming Zhang, and Dong Yu. 2023. On the dimensionality of sentence embeddings. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 10344–10354, Singapore. Association for Computational Linguistics.

Yile Wang, Zhanyu Shen, and Hui Huang. 2025. LDIR: Low-dimensional dense and interpretable text embeddings with relative representations. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 14397–14409, Vienna, Austria. Association for Computational Linguistics.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*.

An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, et al. 2024. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*.

Bowen Zhang, Kehua Chang, and Chunping Li. 2024. Simple techniques for enhancing sentence embeddings in generative language models. In *International Conference on Intelligent Computing*, pages 52–64.

Yanzhao Zhang, Mingxin Li, Dingkun Long, Xin Zhang, Huan Lin, Baosong Yang, Pengjun Xie, An Yang, Dayiheng Liu, Junyang Lin, et al. 2025. Qwen3 embedding: Advancing text embedding and reranking through foundation models. *arXiv preprint arXiv:2506.05176*.

Yeqin Zhang, Yunfei Wang, Chen Jiaxuan, Ke Qin, Yizheng Zhao, and Cam-Tu Nguyen. 2026. LLM-based embeddings: Attention values encode sentence semantics better than hidden states. In *Forty-third International Conference on Machine Learning*.

## A Details of Baselines

We compare our method against following baselines: **PromptEOL** (Jiang et al., 2024), a standard hidden-state-based prompt approach; **Token Prepending** (Fu et al., 2025), a training-free method that prepends tokens to mitigate causal-attention bias; **Contrastive Prompting** (Cheng et al., 2025), an inference-time method that steers semantic encoding with contrastive prompts; **LLM2Vec** (BehnamGhader et al., 2024), an unsupervised method that converts decoder-only LLMs into strong text encoders by enabling bidirectional attention, masked next-token prediction, and contrastive learning; **SemPA** (Chen et al., 2026), a semantic preference alignment method that improves sentence representations with sentence-level DPO; **AlignedWVA** (Zhang et al., 2026), a training-free LLM-based embedding method that leverages attention outputs vectors for sentence representation and serves as our main attention-based baseline. Moreover, we compare with classical BERT-based sentence embedding baselines: **Sentence-BERT** (Reimers and Gurevych, 2019), which employs Siamese and triplet network structures to derive semantically meaningful sentence embeddings; **SimCSE** (Gao et al., 2021), a simple contrastive learning framework that learns sentence embeddings using dropout-based augmentation in the unsupervised setting and NLI supervision in the supervised setting; and **PromptBERT** (Jiang et al., 2022), a prompt-based contrastive learning method that improves BERT sentence embeddings by reformulating sentence representation with prompts and template denoising.

## B Example of Datasets

Table 4 provides representative sentence pairs from the evaluation datasets, with dataset sizes shown in parentheses.

| Type | Content |
|---|---|
| | **STS12 (3108)** |
| **Sentence1** | the problem likely will mean corrective changes before the shuttle fleet starts flying again . |
| **Sentence2** | he said the problem needs to be corrected before the space shuttle fleet is cleared to fly again . |
| **Score** | 4.4 |
| **Link** | https://huggingface.co/datasets/mteb/sts12-sts |
| | **STS13 (1500)** |
| **Sentence1** | this frame contains words that describe an item 's static position on a scale with respect to some property variable . |
| **Sentence2** | lacking in specific resources , qualities or substances ; |
| **Score** | 0.4 |
| **Link** | https://huggingface.co/datasets/mteb/sts13-sts |
| | **STS14 (3750)** |
| **Sentence1** | how about some testimonies from real health experts ? |
| **Sentence2** | also , who 's to say there aren 't testimonies from " real " health experts on there ? |
| **Score** | 3 |
| **Link** | https://huggingface.co/datasets/mteb/sts14-sts |
| | **STS15 (3000)** |
| **Sentence1** | you 'll need to check the particular policies of each publisher to see what is allowed and what is not allowed . |
| **Sentence2** | if you need to publish the book and you have found one publisher that allows it . |
| **Score** | 3 |
| **Link** | https://huggingface.co/datasets/mteb/sts15-sts |
| | **STS16 (3108)** |
| **Sentence1** | what is the best way to store fresh berries ? |
| **Sentence2** | what is the best way to fix this garage floor ? |
| **Score** | 0 |
| **Link** | https://huggingface.co/datasets/mteb/sts16-sts |
| | **STSBenchmark (1379)** |
| **Sentence1** | A girl is styling her hair. |
| **Sentence2** | A girl is brushing her hair. |
| **Score** | 2.5 |
| **Link** | https://huggingface.co/datasets/mteb/stsbenchmark-sts |
| | **STS-R (9927)** |
| **Sentence1** | A group of kids is playing in a yard and an old man is standing in the background |
| **Sentence2** | A group of boys in a yard is playing and a man is standing in the background |
| **Score** | 4.5 |
| **Link** | https://huggingface.co/datasets/mteb/sickr-sts |

Table 4: Examples of sentence pairs and similarity scores from the STS evaluation datasets.

## C Implementation Details

For the main experiments, our method uses a fixed Semantic Component $A_n^L$ and a selection budget of the top-1024 semantic dimensions across all models. The remaining configurations, namely the prompt setting, semantic vector, and the number of languages, are detailed as follows. Specifically, the English prompt applies a shared English template across all translations, whereas the language-specific prompt tailors the template to the native language of each target translation. For semantic vector construction, source denotes using the representation from the original input text, while mean denotes averaging the representations across the selected languages. The language count indicates how many languages are utilized for semantic dimension selection. As shown in the Table 5, the optimal configuration varies by model, suggesting that different model families and instruction-tuning approaches exhibit distinct degrees of multilingual representation stability. Generally, the combination of language-specific prompt and mean semantic vectors is frequently selected, indicating that aggregating multilingual representations helps identify more robust semantic dimensions.

| Models & Links | Prompt Setting | Semantic Vector | Language Counts |
|---|---|---|---|
| LLaMA2-7B | English | Source | 9 |
| | Language-Specific | Mean | 4 |
| LLaMA3.1-8B | English | Source | 11 |
| | Language-Specific | Mean | 6 |
| Qwen2.5-7B | English | Mean | 11 |
| | Language-Specific | Mean | 7 |
| Qwen3-8B | English | Mean | 12 |
| | Language-Specific | Mean | 12 |
| LLaMA2-7B-it | English | Source | 12 |
| | Language-Specific | Mean | 6 |
| LLaMA3.1-8B-it | English | Mean | 12 |
| | Language-Specific | Mean | 10 |
| Qwen2.5-7B-it | English | Mean | 11 |
| | Language-Specific | Mean | 8 |
| Qwen3-8B-it | English | Mean | 12 |
| | Language-Specific | Mean | 10 |
| Phi3-mini-it | English | Mean | 8 |
| | Language-Specific | Mean | 4 |
| Phi3.5-mini-it | English | Mean | 10 |
| | Language-Specific | Mean | 8 |

Table 5: Detailed parameter settings for the main experiments across different models, including the chosen prompt setting, semantic vector construction method, and language counts.

# D Additional Experimental Results

## D.1 Validity of Semantic Components

Figure 10 shows the complementary results on the remaining six models. The same pattern holds consistently across these models: prompt-selected semantic components substantially outperform random components in all evaluated settings. For example, on Qwen3-8B, language-specific selection improves the average score from 53.41 to 80.53; on LLaMA2-7B-it, it improves the score from 56.02 to 77.16. The gap is especially pronounced for the Qwen2.5 family, where random components obtain an average score of around 42, whereas semantic components reach nearly 79. Similar improvements are also observed for the LLaMA3.1 and Phi models. These results further confirm that the selected components capture task-relevant semantic information, rather than merely benefiting from random feature sparsification.

## D.2 Semantic Components Selection

Figure 11 provides the complementary component-selection results for the remaining six models. The overall trend is consistent with the main analysis: the cumulative attention outputs ($A_n^L$) are usually the strongest representation for text semantic similarity. The advantage of $A_n^L$ is especially clear for LLaMA3.1-8B, LLaMA3.1-8B-it, Phi3-mini-it, and Phi3.5-mini-it, where it outperforms the alternatives on every evaluated STS task. The only substantial exceptions come from the Qwen2.5 family, where $h_n^L$ is competitive and occasionally better on tasks such as STS14, STS15, STS16, and STS-B. Even in these cases, $F_n^L$ remains consistently weaker. This reinforces the conclusion that $A_n^L$ is the most reliable representation for exposing semantic similarity information.

## D.3 Language Selection Strategies

The appendix provides the full set of language-selection results omitted from the main text. Figure 12 extends the language-wise ranking analysis to the remaining models. The overall pattern is consistent with Figure 5: English remains the strongest individual language in all cases, and French and Spanish are usually placed near the top. At the same time, the middle-ranked languages differ noticeably across models. For example, Qwen2.5-7B ranks Chinese and German relatively high, LLaMA3.1-8B assigns a high rank to Japanese, while the Phi models favor Portuguese and Italian among the top candidates. This confirms that the ranking is not universal, and that the language pool should be selected separately for each model.

Figure 13 reports the two complementary configurations for the main models. With the English prompt and mean vector, performance remains sensitive to the number of selected languages: LLaMA2 models reach their best scores with small pools, while Qwen models generally benefit from larger pools and often peak around $m = 11$ or $m = 12$. In contrast, the language-specific prompt with source vector produces much flatter curves. This suggests that source-vector aggregation is more robust to the inclusion of additional languages, whereas mean-vector aggregation can either amplify useful multilingual signals or introduce noise depending on the model.

Figures 14 and 15 show the same comparisons on the remaining models. The language-specific prompt with mean vector again exhibits the strongest dependence on $m$: Phi3-mini-it peaks early at $m = 4$, LLaMA3.1-8B peaks around $m = 6$, while LLaMA3.1-8B-it benefits from a larger pool and peaks around $m = 10$. The English prompt with source vector and the language-specific prompt with source vector are comparatively stable, with only small changes as $m$ increases. These additional results reinforce the main conclusion that careful language selection is most

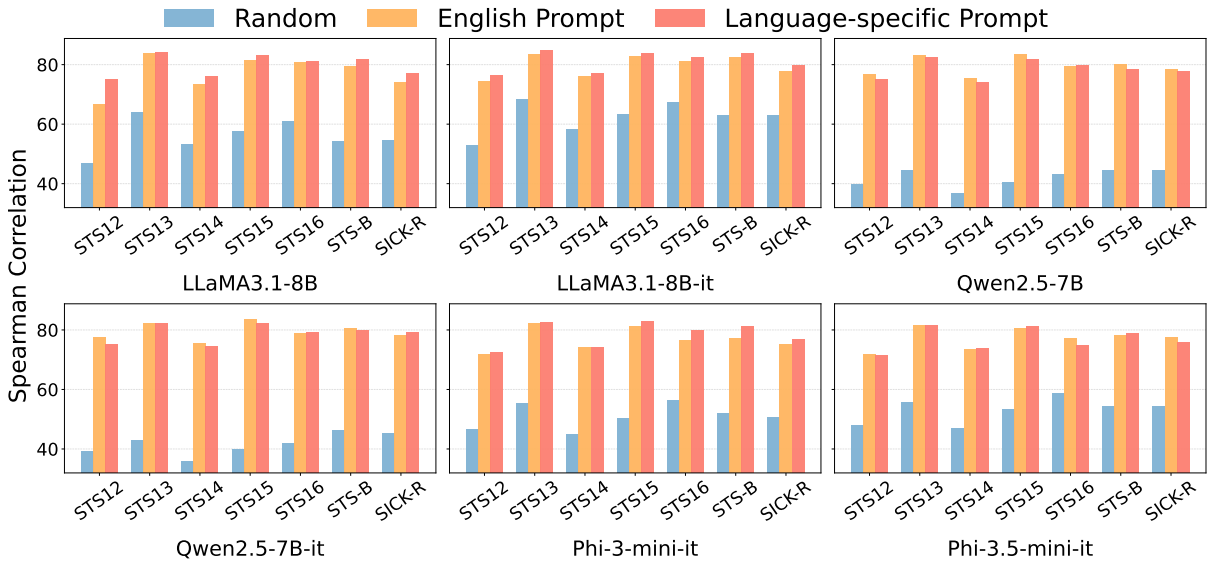



Figure 10: STS performance comparison between random components and semantic components on the remaining models.

important when using mean-vector aggregation, while source-vector settings are less sensitive but still show model-dependent optima.

### D.4 Impacts of Prompts and Semantic Vector

Figure 16 reports the remaining four models. The same pattern is largely preserved: the language-specific prompt with the mean vector obtains the best result on three of the four models, including LLaMA3.1-8B, LLaMA3.1-8B-it, and Phi3-mini-it. Averaged over these remaining models, this setting reaches 79.05, outperforming the English-prompt mean-vector setting by 1.77 points and the language-specific source-vector setting by 2.40 points. The only exception is Phi3.5-mini-it, where the English prompt with the mean vector is slightly better by 0.47 points, suggesting that the advantage of localized prompts is not uniform across all model families.

Figure 17 further breaks down the comparison across STS tasks. The language-specific prompt with the mean vector achieves the highest average score on six of the seven evaluated tasks, while the language-specific source vector setting is best only on STS16. Across tasks, language-specific prompts improve the mean vector by 1.24 points on average, with consistent gains on STS12, STS13, STS14, STS15, STSBenchmark, and SICK-R. In contrast, under the source vector setting, the effect of prompt language remains marginal, with task-level differences staying within 0.30 points. These results reinforce the main finding that prompt localization is most beneficial when combined with mean-vector construction, where multilingual representations are explicitly aggregated before scoring.

### D.5 Attention Aggregation Strategies

Figures 18 and 19 provide supplementary comparisons across the remaining prompt and vector configurations. The trends observed in the main text remain consistent: cumulative attention systematically achieves a higher peak average performance compared to layer-specific attention across all evaluated settings. Specifically, LLaMA2-7B shows an improvement of 4.97 points when using the English prompt combined with source vectors, 2.31 points with the English prompt and mean vectors, and 5.17 points under the language-specific prompt combined with source vectors. Correspondingly, Qwen3-8B-it exhibits gains of 4.50, 3.29, and 4.21 points under the same respective configurations. These results demonstrate that the advantages of cumulative aggregation are robust and independent of specific prompt choices or value-vector constructions.

Figures 20 and 21 further detail the performance of both strategies on the STSBenchmark and SICK-R tasks. Cumulative attention consistently yields superior task-specific peaks across nearly all settings. For instance, when applying the English prompt with source vectors to LLaMA2-7B, cumulative attention boosts performance on STSBenchmark from 75.89 to 80.99, and on SICK-R from 73.22 to 77.12. Similarly, employing the language-

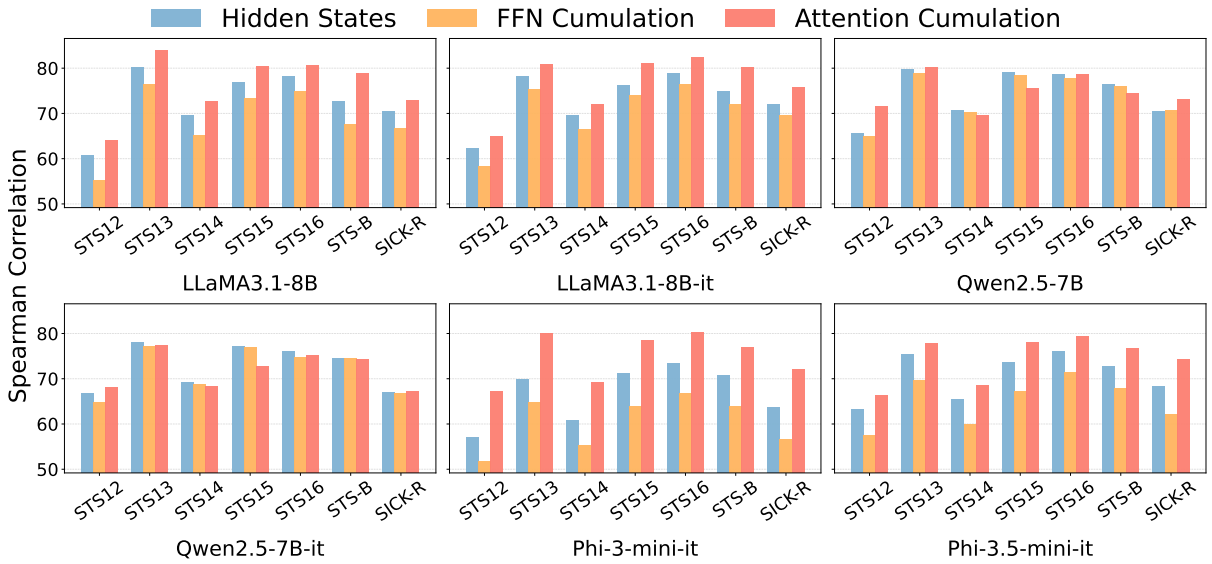



Figure 11: Task-level semantic similarity performance of different internal representations on remaining models.

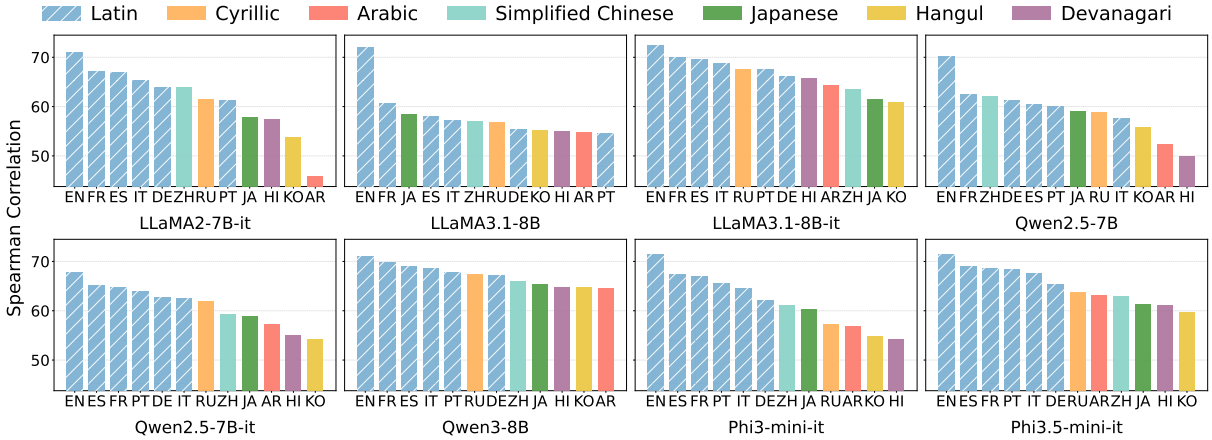



Figure 12: Language-wise spearman correlation of cumulative attention on remaining models.

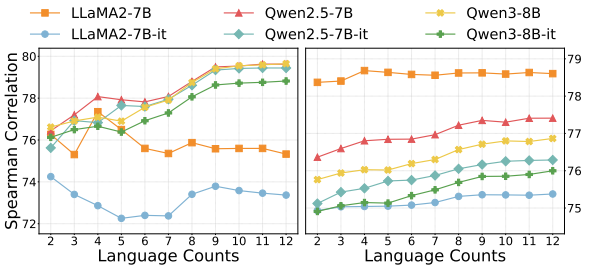



Figure 13: Analysis of language selection strategies on English prompt with mean vector (left) and source vector (right). Comparison across different top-$m$ language selections.

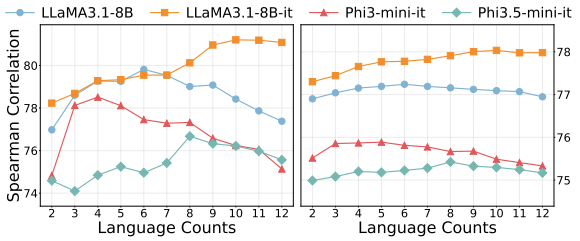



Figure 14: Analysis of language selection strategies on language-specific prompt with mean vector (left) and source vector (right) on remaining models. Comparison across different top-$m$ language selections.

specific prompt with mean vectors on Qwen3-8B-it elevates STSBenchmark from 79.06 to 82.82, and SICK-R from 75.19 to 78.68. Ultimately, these supplementary findings reinforce the conclusion that cumulative attention more effectively preserves valuable cross-layer semantic signals, generating more stable representations than those derived from any single layer.

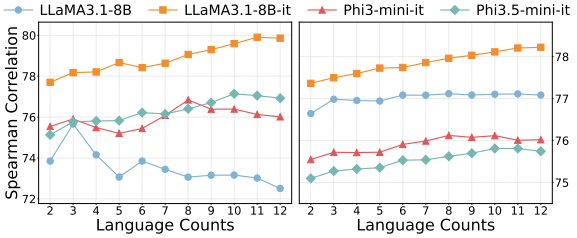



Figure 15: Analysis of language selection strategies on English prompt with mean vector (left) and source vector (right) on remaining models. Comparison across different top-$m$ language selections.

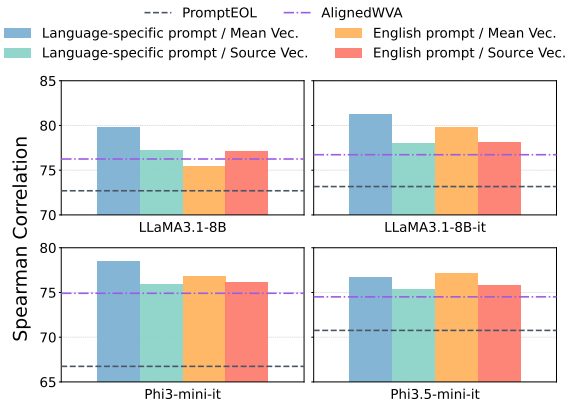



Figure 16: Comparison of English and language-specific prompts under source and mean vectors settings on remaining models.

### D.6 Impacts Semantic Dimension Sizes

We provide additional results for the remaining prompt and vector configurations in Figure 22. The same overall conclusion holds: the retained dimension budget is primarily controlled by the prompt language. For the English prompt combined with a mean vector, performance increases steadily from $k = 256$ to $k = 1024$, with only marginal change afterwards. This indicates that English prompts still benefit from a relatively larger semantic subspace even when the vector is averaged across languages. In contrast, the configuration using a language-specific prompt combined with a source vector reaches near-optimal performance at $k = 512$ or $k = 768$ and then remains almost unchanged. This further supports that language-aligned prompts allow semantic information to be concentrated in a compact set of dimensions.

Figures 24 and 24 show the same analysis on the remaining models. Although the absolute scores vary across model families, the trend is consistent. Configurations using language-specific prompts saturate quickly, usually after $k = 512$ or $k = 768$, while configurations relying on English prompts show more gradual improvements as $k$ increases and often perform best at $k = 1024$ or $k = 1280$. These results confirm that the observed dimension-size behavior is not specific to the main set of models, but generalizes across additional model families.

## E Effectiveness Across Different Prompts

To further examine whether the identified semantic components are robust to different prompting strategies, we evaluate our method under two other commonly used prompts, namely **Pretended CoT** and **Knowledge Enhancement** (Zhang et al., 2024). The results are reported in Table 6. Overall, our method consistently achieves the best or second-best average performance across different models and prompts, showing that the semantic information uncovered by DySEM is not tied to a specific prompt format.

Under the Pretended CoT prompt, DySEM brings consistent improvements over both the original prompt-based representations and AlignedWVA. On LLaMA2-7B, our method achieves average scores of 80.40 and 80.26 with English and language-specific prompts, respectively, outperforming AlignedWVA by 0.60 and 0.46 points. The gains are more pronounced on Qwen3-8B, where the language-specific prompt version reaches an average score of 80.25, improving over AlignedWVA by 8.39 points. Similar improvements are also observed on instruction-tuned models. For example, on Qwen3-8B-it, our method obtains average scores of 80.11 and 80.16, surpassing AlignedWVA by more than 5.5 points.

Under the Knowledge Enhancement prompt, our method also maintains strong performance. On LLaMA2-7B, DySEM achieves the best average score of 80.68 with the English prompt and 80.52 with the language-specific prompt, both outperforming AlignedWVA. On Qwen3-8B, our English-prompt variant obtains an average score of 78.08, exceeding AlignedWVA by 2.87 points. For instruction-tuned models, the improvements remain stable: on LLaMA2-7B-it, our method reaches 76.64 and 76.27 under the two prompt variants, while on Qwen3-8B-it it achieves 78.93 and 79.20, clearly outperforming the corresponding AlignedWVA results.

These results demonstrate that DySEM is effective across different prompt designs. Although the

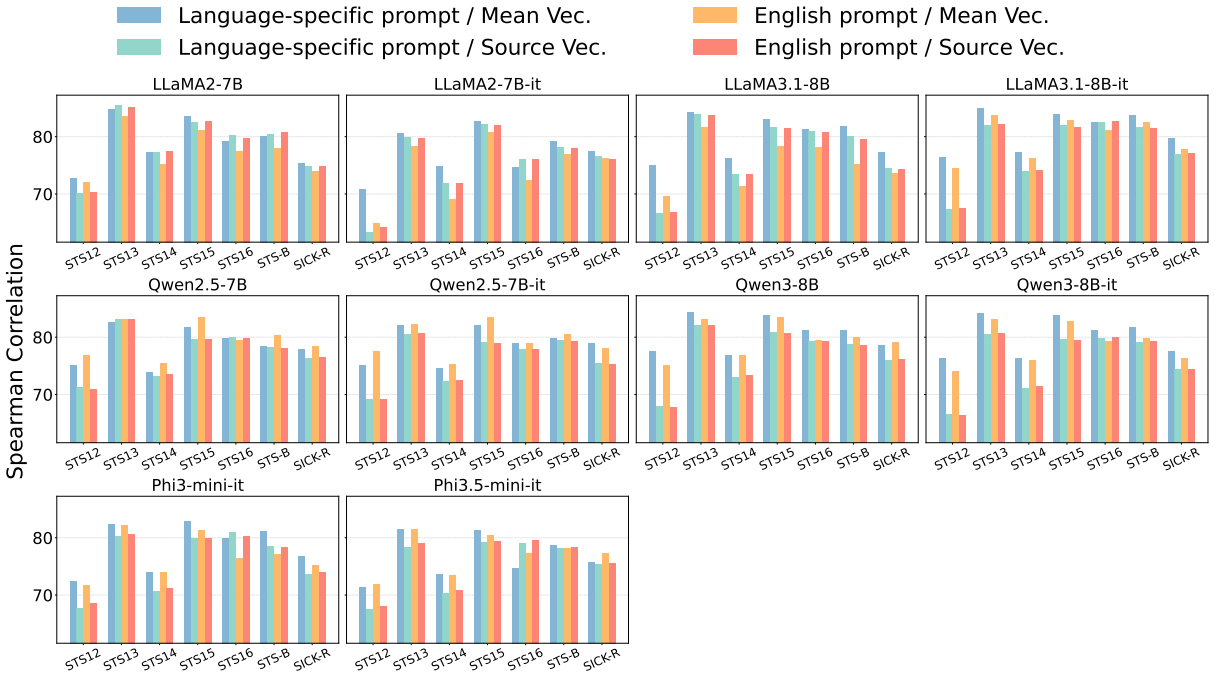



Figure 17: Comparison of English and language-specific prompts under source and mean vectors settings across STS tasks.

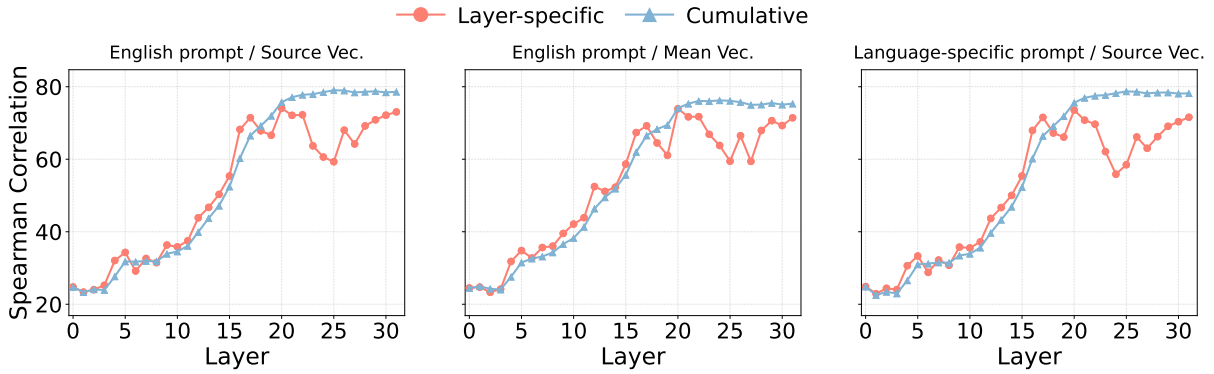



Figure 18: Average performance across tasks on LLaMA2-7B with remaining setting. Comparison between layer-specific attention and cumulative attention.

absolute performance may vary between English and language-specific prompts, our method consistently improves STS performance over prompt-only representations and AlignedWVA. This suggests that the semantic subspaces discovered by DySEM capture prompt-agnostic semantic knowledge inside LLMs, enabling more reliable semantic computation under diverse prompting conditions.

## F Transferability to MTEB Datasets

To examine whether the semantic subspaces identified by DySEM are useful beyond pairwise semantic textual similarity, we further evaluate them on representative retrieval and clustering tasks from the Massive Text Embedding Benchmark (MTEB). Specifically, we report the Spearman correlation on STS15, the average nDCG@10 score across the SciFact and ArguAna retrieval datasets, and the V-measure on the TwentyNewsgroupsClustering task. We compare DySEM with AlignedWVA under the same models. The results are summarized in Table 7.

DySEM consistently improves STS15 performance for both models, and the gains also transfer to retrieval and clustering tasks. On LLaMA2-7B, DySEM improves the average retrieval score from 44.88 to 46.10, corresponding to an absolute gain of 1.22 points. It also substantially improves the TwentyNewsgroupsClustering V-measure from 33.82 to

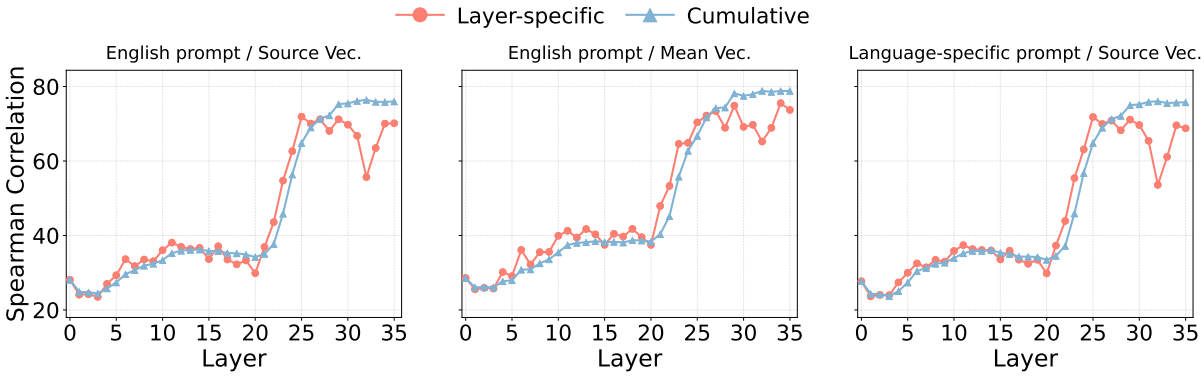



Figure 19: Average performance across tasks on Qwen3-8B-it with remaining settings. Comparison between layer-specific attention and cumulative attention.

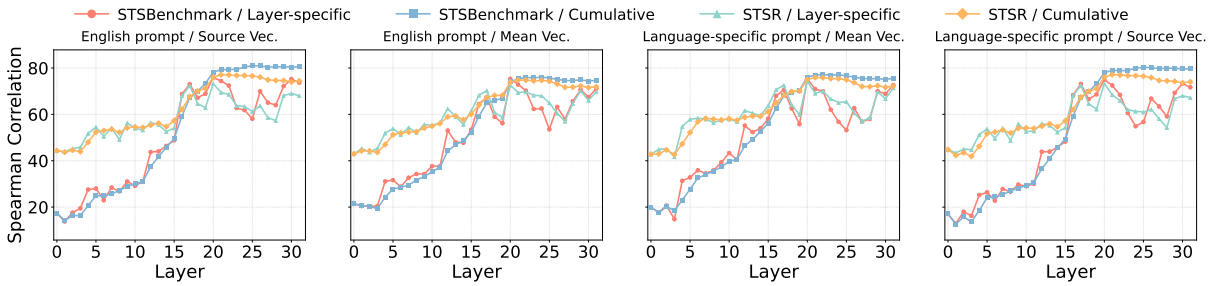



Figure 20: Task-wise performance on STSBenchmark and SICK-R for LLaMA2-7B with four settings. Comparison between layer-specific attention and cumulative attention.

| | Methods | #Dim. | STS12 | STS13 | STS14 | STS15 | STS16 | STS-B | SICK-R | Avg. |
|---|---|---|---|---|---|---|---|---|---|---|
| LLaMA2-7B | **Pretended CoT (Zhang et al., 2024)** | 4096 | 61.00 | 79.62 | 69.42 | 74.78 | 77.20 | 72.11 | 70.56 | 72.10 |
| | **Pretended CoT + AlignedWVA (Zhang et al., 2026)** | 4096 | 69.81 | 86.37 | 77.83 | 83.63 | **82.80** | 82.07 | 76.10 | 79.80 |
| | **Pretended CoT + (English Prompt, Ours)** | 1294* | 71.11 | **86.67** | **79.05** | **84.21** | 82.70 | **82.56** | **76.51** | **80.40** |
| | **Pretended CoT + (Language-specific Prompt, Ours)** | 999* | **71.86** | 86.55 | 78.66 | 84.15 | 82.38 | 81.92 | 76.29 | 80.26 |
| | **Knowledge Enhancement (Zhang et al., 2024)** | 4096 | 61.94 | 80.37 | 71.40 | 78.43 | 77.24 | 76.57 | 75.77 | 74.53 |
| | **Knowledge Enhancement + AlignedWVA (Zhang et al., 2026)** | 4096 | 68.12 | 86.13 | 77.29 | 83.02 | **81.84** | 83.11 | 78.39 | 79.70 |
| | **Knowledge Enhancement + (English Prompt, Ours)** | 1117* | 70.69 | 86.45 | **78.98** | **84.14** | 81.80 | **83.91** | **78.79** | **80.68** |
| | **Knowledge Enhancement + (Language-specific Prompt, Ours)** | 908* | **70.90** | **86.57** | 78.50 | 83.72 | 81.80 | 83.46 | 78.70 | 80.52 |
| LLaMA2-7B-it | **Pretended CoT (Zhang et al., 2024)** | 4096 | 61.39 | 79.31 | 69.36 | 76.45 | 72.62 | 73.47 | 70.62 | 71.89 |
| | **Pretended CoT + AlignedWVA (Zhang et al., 2026)** | 4096 | 62.37 | 80.45 | 71.17 | 81.02 | **76.26** | 77.86 | 76.05 | 75.02 |
| | **Pretended CoT + (English Prompt, Ours)** | 1132* | 64.51 | 80.17 | 72.85 | 81.98 | 76.11 | 78.74 | 76.59 | 75.85 |
| | **Pretended CoT + (Language-specific Prompt, Ours)** | 837* | **72.22** | **81.73** | **76.03** | **83.14** | 75.17 | **79.95** | **76.72** | **77.85** |
| | **Knowledge Enhancement (Zhang et al., 2024)** | 4096 | 57.55 | 79.32 | 68.91 | 74.36 | 74.06 | 73.18 | 72.63 | 71.43 |
| | **Knowledge Enhancement + AlignedWVA (Zhang et al., 2026)** | 4096 | 62.09 | 82.77 | 71.96 | 79.34 | **77.62** | 77.51 | 75.63 | 75.27 |
| | **Knowledge Enhancement + (English Prompt, Ours)** | 1034* | **65.92** | 82.68 | **74.25** | **81.27** | 77.49 | **78.71** | 76.15 | **76.64** |
| | **Knowledge Enhancement + (Language-specific Prompt, Ours)** | 946* | 64.85 | **83.15** | 73.51 | 80.49 | 77.39 | 78.31 | **76.22** | 76.27 |
| Qwen3-8B | **Pretended CoT (Zhang et al., 2024)** | 4096 | 56.78 | 73.17 | 58.71 | 67.99 | 73.21 | 68.71 | 61.99 | 65.79 |
| | **Pretended CoT + AlignedWVA (Zhang et al., 2026)** | 4096 | 59.12 | 79.37 | 68.50 | 73.34 | 78.85 | 73.30 | 70.53 | 71.86 |
| | **Pretended CoT + (English Prompt, Ours)** | 1007* | 73.02 | 83.26 | 75.77 | 82.05 | 79.02 | 78.57 | 77.97 | 78.52 |
| | **Pretended CoT + (Language-specific Prompt, Ours)** | 752* | **76.40** | **84.42** | **76.56** | **83.68** | **81.64** | **80.63** | **78.43** | **80.25** |
| | **Knowledge Enhancement (Zhang et al., 2024)** | 4096 | 58.28 | 79.15 | 67.73 | 75.00 | 77.14 | 72.36 | 72.37 | 71.72 |
| | **Knowledge Enhancement + AlignedWVA (Zhang et al., 2026)** | 4096 | 64.89 | 82.16 | 73.87 | 80.11 | **79.33** | 72.61 | 73.51 | 75.21 |
| | **Knowledge Enhancement + (English Prompt, Ours)** | 914* | **73.22** | 82.97 | 75.45 | **81.43** | 78.82 | **76.93** | **77.75** | **78.08** |
| | **Knowledge Enhancement + (Language-specific Prompt, Ours)** | 696* | 67.96 | **83.35** | **75.47** | 81.12 | 79.13 | 74.58 | 74.90 | 76.65 |
| Qwen3-8B-it | **Pretended CoT (Zhang et al., 2024)** | 4096 | 52.49 | 64.14 | 54.19 | 70.53 | 74.32 | 71.10 | 62.90 | 64.24 |
| | **Pretended CoT + AlignedWVA (Zhang et al., 2026)** | 4096 | 64.64 | 77.96 | 68.67 | 79.52 | 80.01 | 78.01 | 72.99 | 74.54 |
| | **Pretended CoT + (English Prompt, Ours)** | 1034* | **75.64** | 83.50 | **76.86** | **84.16** | 81.03 | **81.87** | 77.70 | 80.11 |
| | **Pretended CoT + (Language-specific Prompt, Ours)** | 1240* | 75.43 | **84.67** | 75.49 | 83.97 | **81.95** | 81.64 | **77.95** | **80.16** |
| | **Knowledge Enhancement (Zhang et al., 2024)** | 4096 | 59.61 | 73.26 | 63.36 | 74.70 | 75.37 | 73.33 | 71.16 | 70.11 |
| | **Knowledge Enhancement + AlignedWVA (Zhang et al., 2026)** | 4096 | 66.31 | 80.97 | 72.61 | 79.44 | 79.23 | 76.86 | 73.71 | 75.59 |
| | **Knowledge Enhancement + (English Prompt, Ours)** | 905* | 74.10 | 83.12 | **76.22** | 82.67 | **79.45** | 79.91 | 77.04 | 78.93 |
| | **Knowledge Enhancement + (Language-specific Prompt, Ours)** | 683* | **74.40** | **83.56** | 76.19 | **82.84** | 79.20 | **80.98** | **77.25** | **79.20** |

Table 6: Results of Pretended CoT and Knowledge Enhancement prompts.

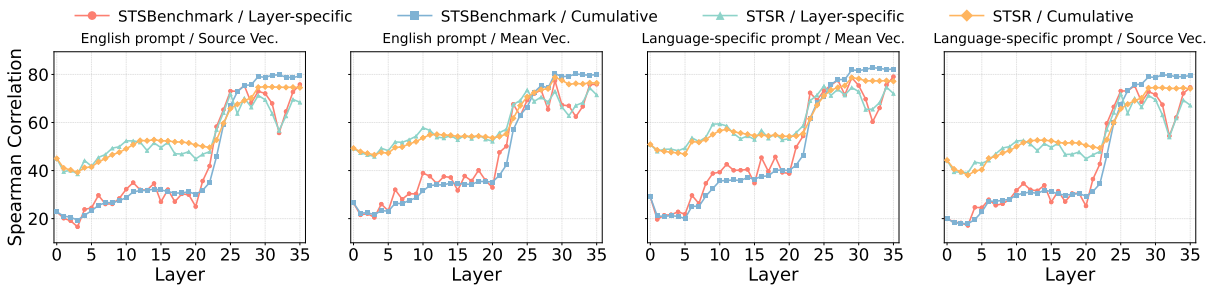



Figure 21: Task-wise performance on STSBenchmark and SICK-R for Qwen3-8B-it with four settings. Comparison between layer-specific attention and cumulative attention.

| Model | Method | STS *Spearman* | Retrieval *nDCG@10* | Clustering *V-measure* |
|---|---|---|---|---|
| **LLaMA2-7B** | **AlignedWVA (Zhang et al., 2026)** | 81.68 | 44.88 | 33.82 |
| | **DySEM (ours)** | **83.47** | **46.10** | **43.83** |
| **Qwen3-8B** | **AlignedWVA (Zhang et al., 2026)** | 77.38 | 44.89 | 44.51 |
| | **DySEM (ours)** | **83.91** | **49.08** | **44.63** |

Table 7: Performance comparison on MTEB datasets. The best result for each model and task is shown in bold.

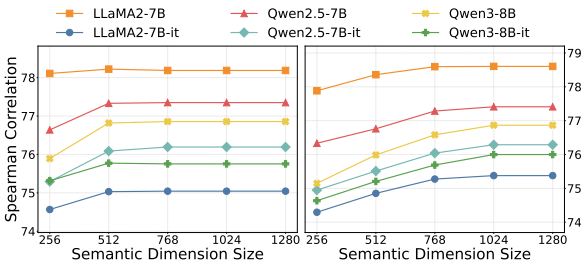



Figure 22: Analysis of semantic dimension sizes on Language-specific prompt (left) and English prompt (right) with source vector. Comparison across different top-$k$ semantic dimension selections.

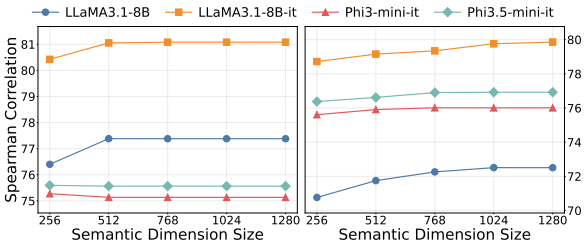



Figure 23: Analysis of semantic dimension sizes on Language-specific prompt (left) and English prompt (right) with mean vector on remaining models. Comparison across different top-$k$ semantic dimension selections.

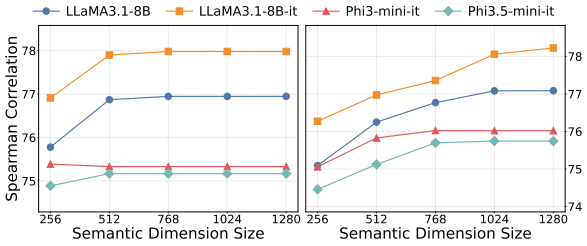



Figure 24: Analysis of semantic dimension sizes on Language-specific prompt (left) and English prompt (right) with source vector on remaining models. Comparison across different top-$k$ semantic dimension selections.

43.83, yielding a gain of 10.01 points. These results indicate that the dynamically selected semantic dimensions capture information that is useful not only for measuring sentence-level similarity, but also for ranking relevant documents and grouping semantically related texts.

On Qwen3-8B, DySEM achieves a 6.53-point improvement on STS15, increasing the score from 77.38 to 83.91. It also improves the average retrieval score from 44.89 to 49.08, corresponding to a gain of 4.19 points. For clustering, DySEM slightly outperforms AlignedWVA. Overall, these results show that DySEM is not limited to sentence-pair similarity. Its dynamically constructed semantic representations transfer to both retrieval and clustering, demonstrating the broader usefulness of the semantic components identified inside LLMs.

## G Use of AI Assistants

AI assistants were used solely for polishing purposes. They did not contribute to the generation of any core technical contents, experimental results, or research conclusions of this work.